

LightLLM & LightX2V

生成式模型推理框架的探索

ModelTC

Overview

理解



LightLLM



文/图/音频->文

LightX2V



文/图/音频->视频

LightTTS



文本->音频

LightKernel

LightCompress

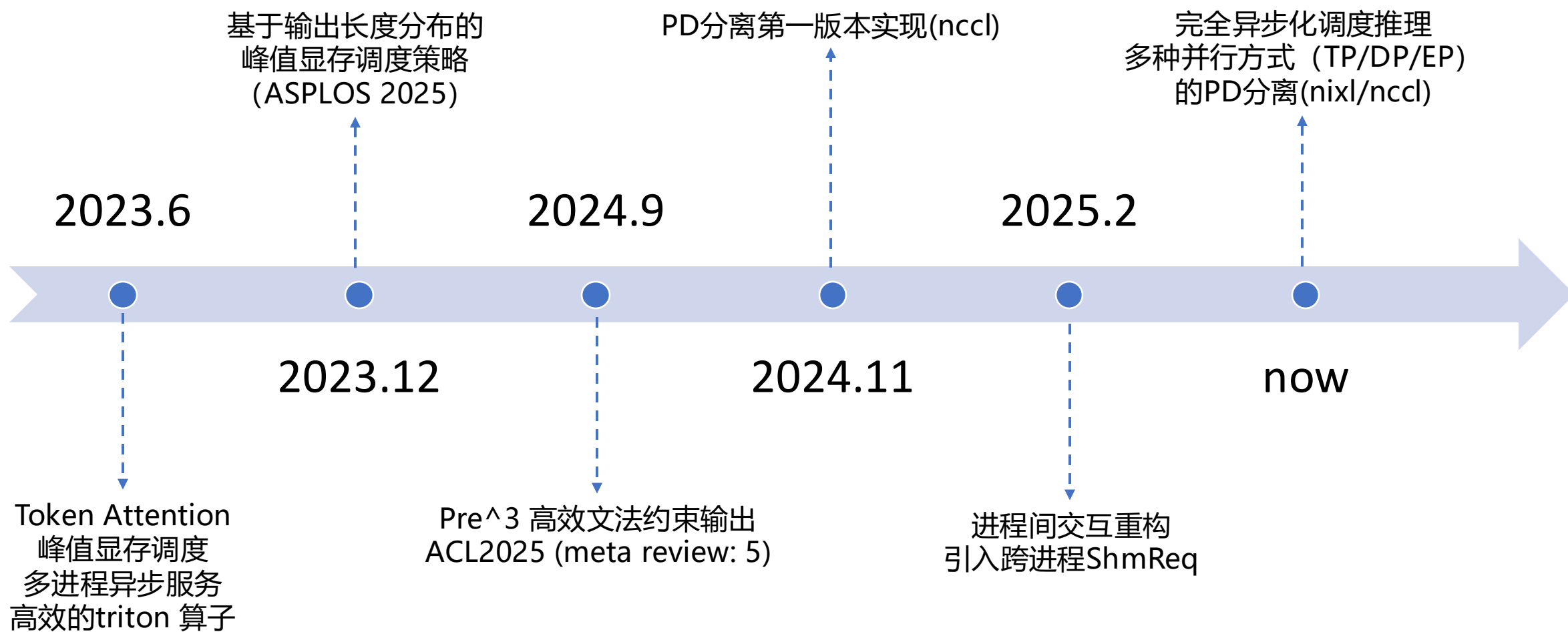
Quantization

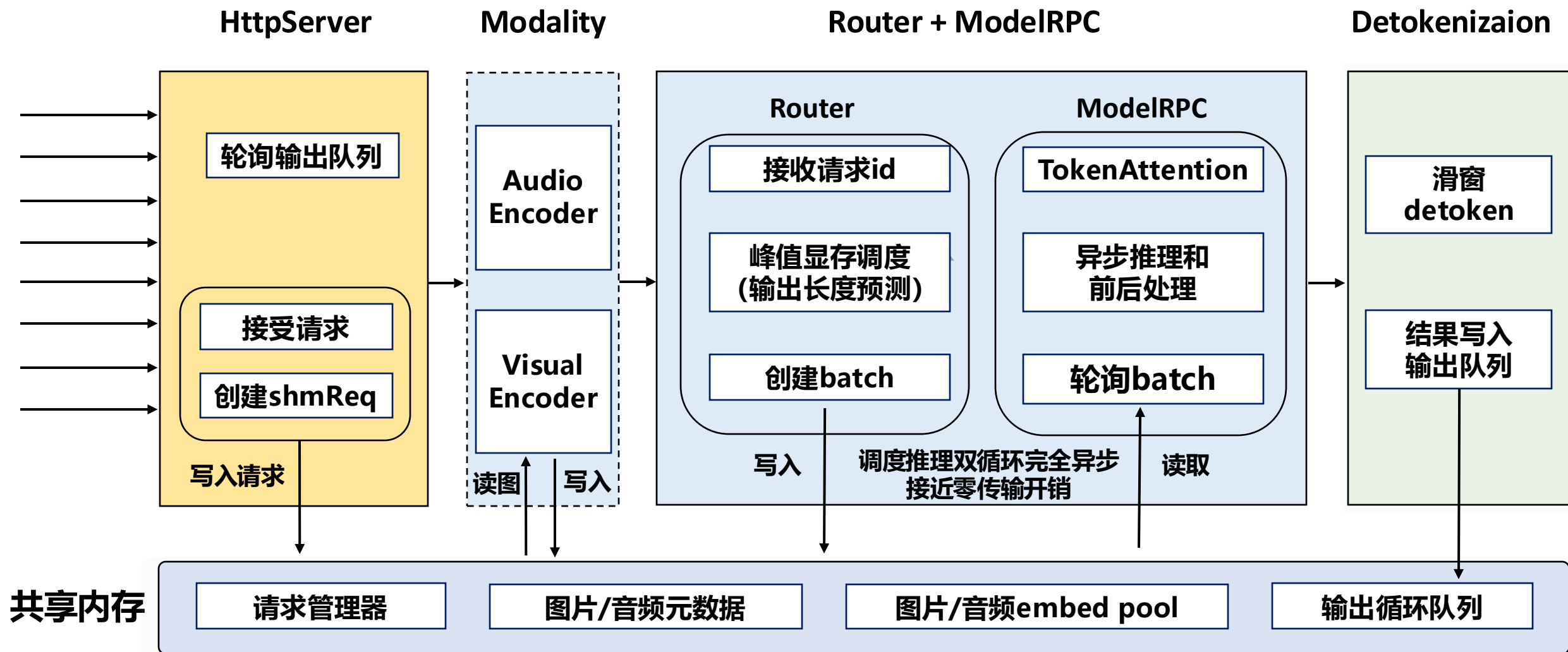
Sparsification

Token Reduction

生成

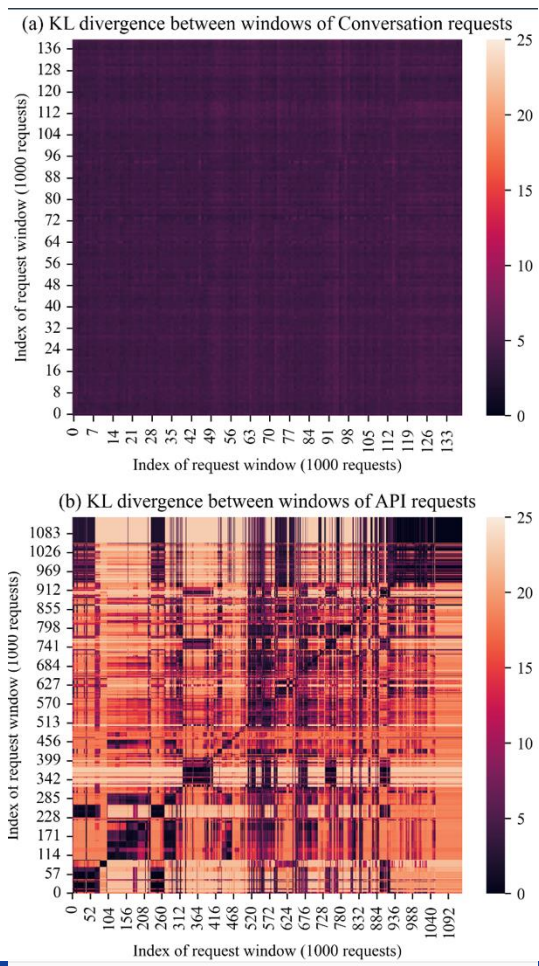






基于长度预测的峰值显存调度

- 历史请求输出分布长度，在相邻的时间窗口，表现出一定的相似度
- 每次调度，都对batch中的请求输出长度进行重新预测。结果从 $P(l > l_t^i)$ 采样，保障请求不被逐出

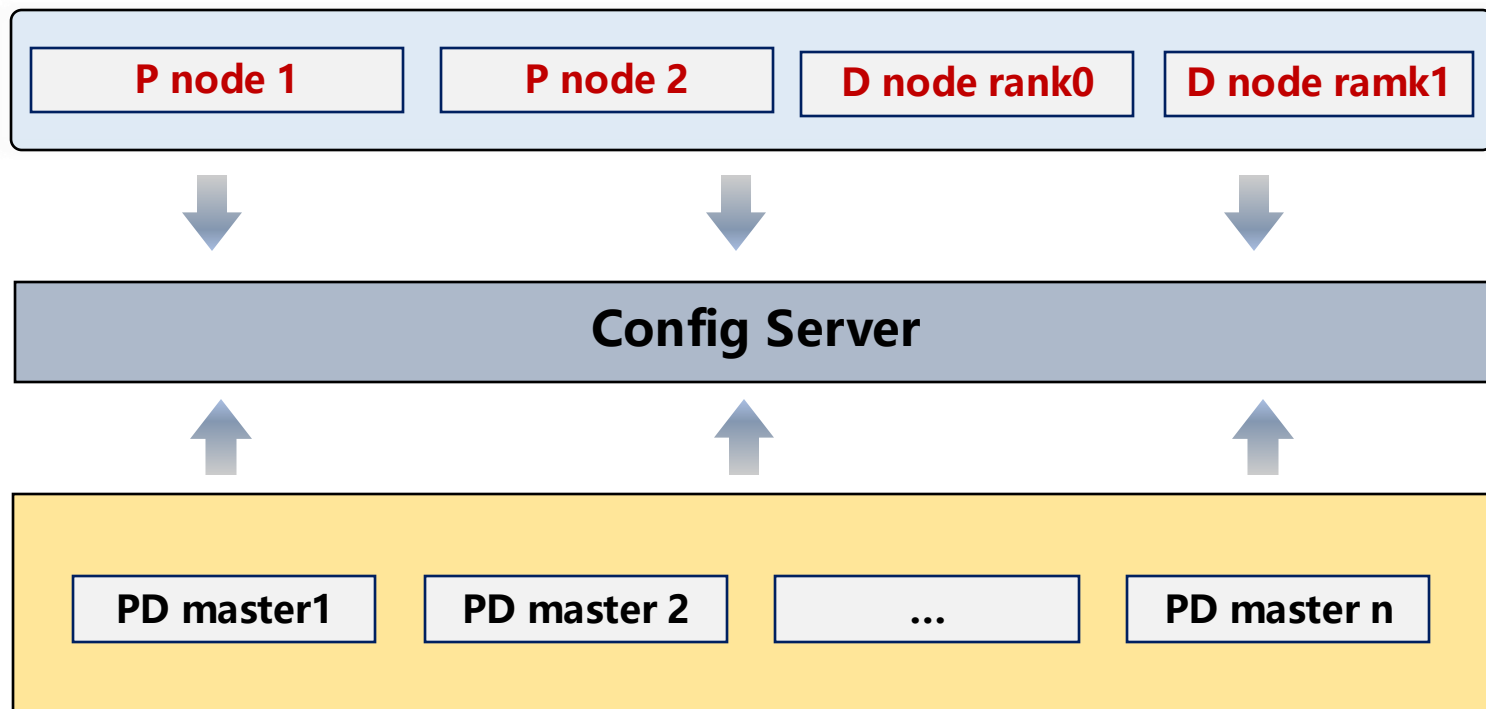


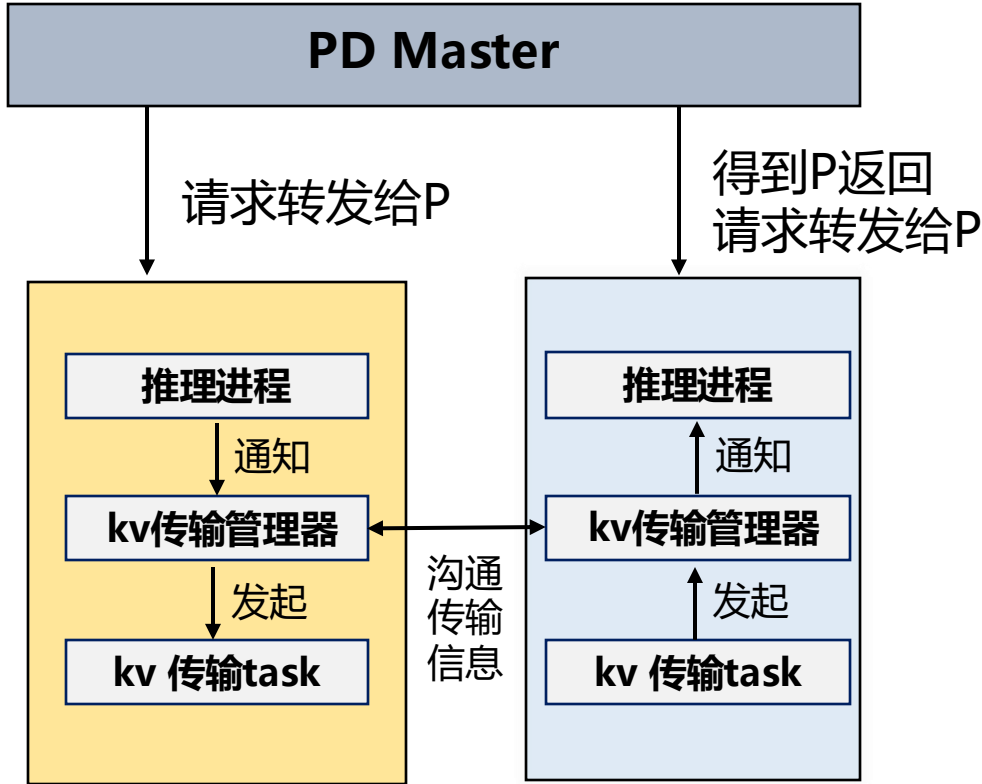
Dataset	Method	Decoding Steps	Memory Utilization	Peak Memory	Evicted Reqs
Decode-heavy dataset-1	Theoretical optimum	300110	94.22%	98.19%	0%
	Past-Future	301730	92.67%	96.56%	6.26%
	Aggressive (water-level=99%)	289690	97.62%	102.18%	91.8%
	Conservative	490980	57.59%	60.66%	0%
Medium dataset-2	Theoretical optimum	692740	91.77%	97.75%	0%
	Past-Future	694080	91.32%	96.18%	6.06%
	Aggressive (water-level=99%)	651800	97.55%	103.06%	94.43%
Prefill-heavy dataset-3	Conservative	878260	72.38%	75.94%	0%
	Theoretical optimum	240270	96.11%	98.14%	0%
	Past-Future	247400	93.38%	95.55%	3.06%
	Aggressive (water-level=99%)	236650	97.58%	99.69%	33.66%
	Conservative	387720	59.56%	61.49%	0%

Table 1. Performance of different scheduling methods on multiple datasets

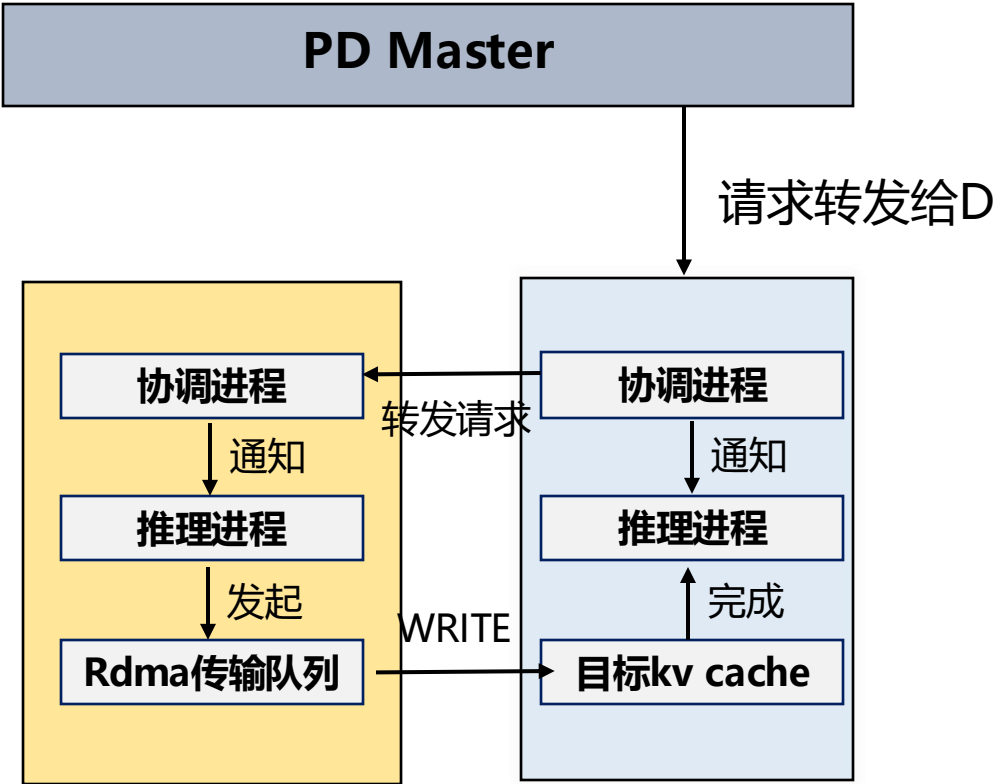
为了便于生产环境的服务启动和动态扩缩容，引入一个CPU 服务 config server

- 启动服务时，pd master, p node, d node 只需传入 config server 的地址，由config server 协助完成 p/d 和master的连接
- 多机服务的通信组建立也由 config server 完成。（例如：ep32的D node）
- 可以方便的对P，D和PD master而进行扩缩容





NCCL方案

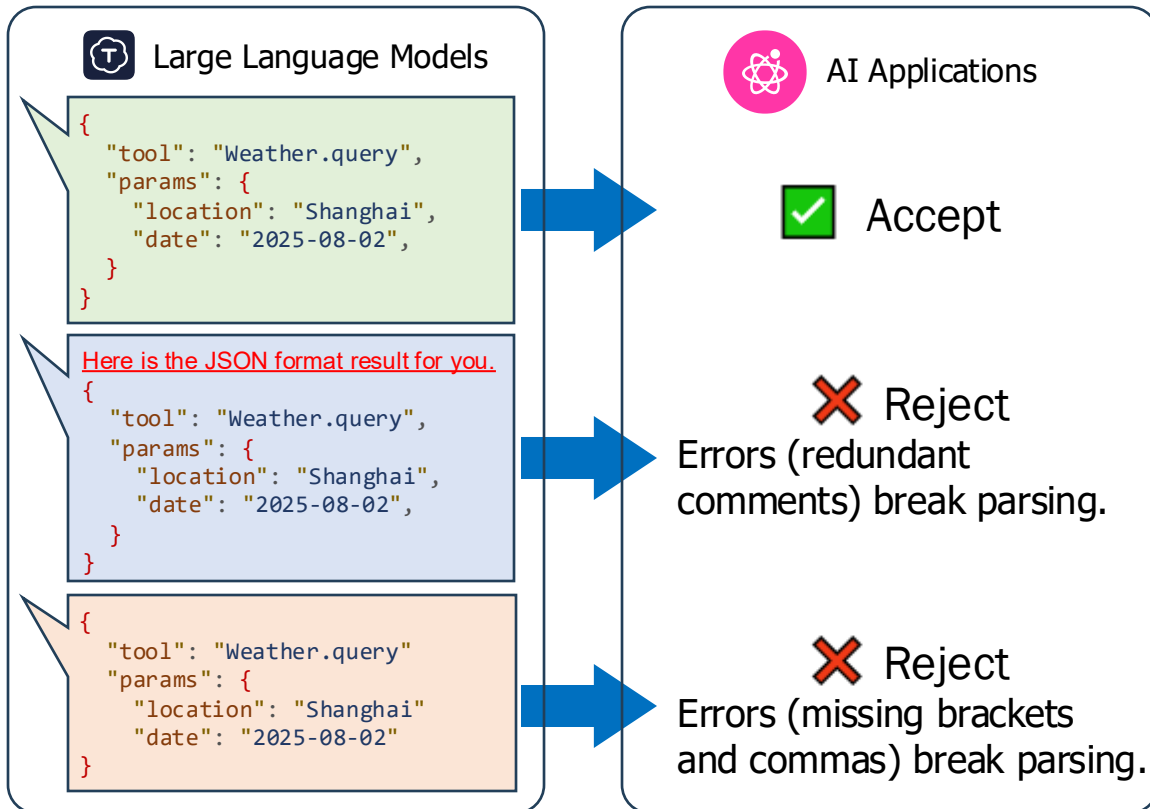


NIXL方案

LLM Structured Generation

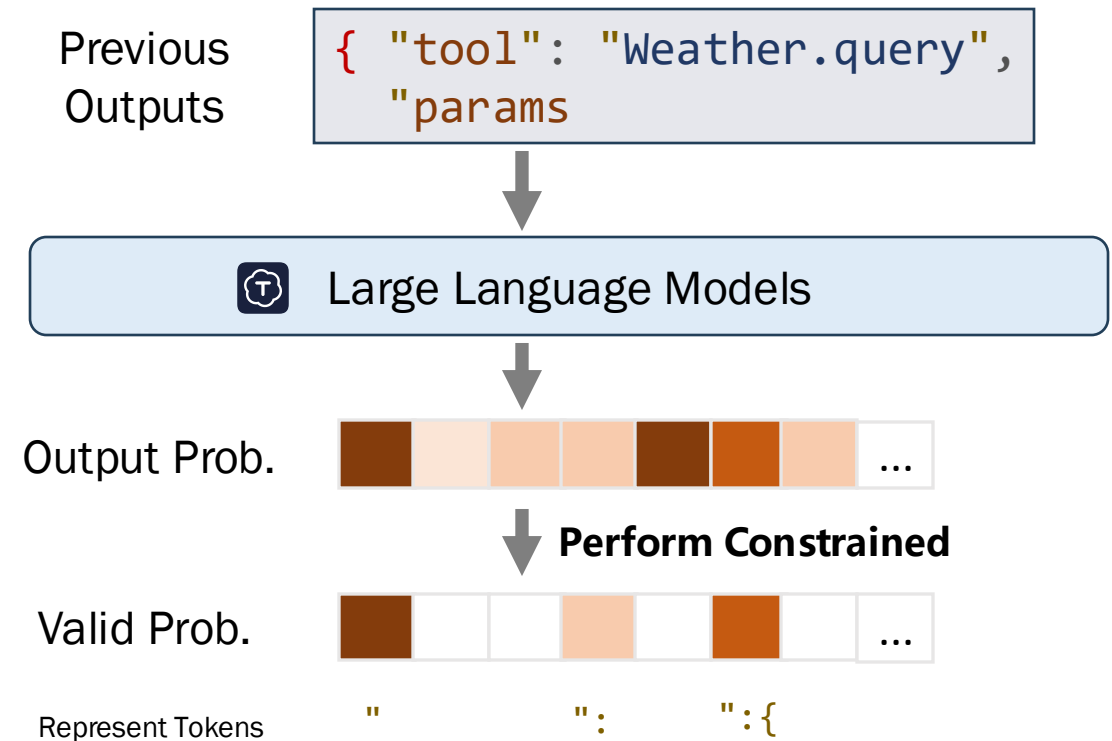
What is Structured Output?

- LLM produces formatted data (JSON, SQL, code) instead of free text
- Enforce strict syntax rules for machine-readable output



How to achieve Structured Output?

- **Constrained Decoding:** Dynamically blocks invalid tokens using probability masks
- **Model Finetuning:** Trains LLM on structured data (JSON/SQL) for rule compliance

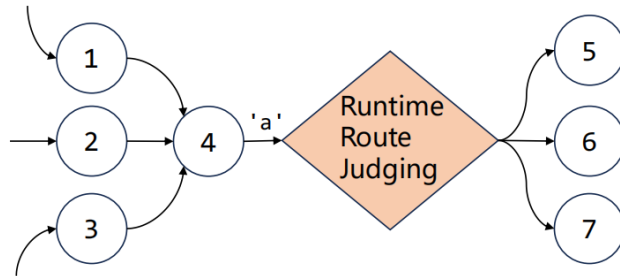


LLM Structured Generation

- Current constrained decoding approaches processing grammars introduces two major performance bottlenecks.

Non-Deterministic Transitions

- Single token may trigger multiple valid grammar paths
- Requires dynamic validation to resolve conflicts
- Slows generation due to decision-making and complex backtrack latency

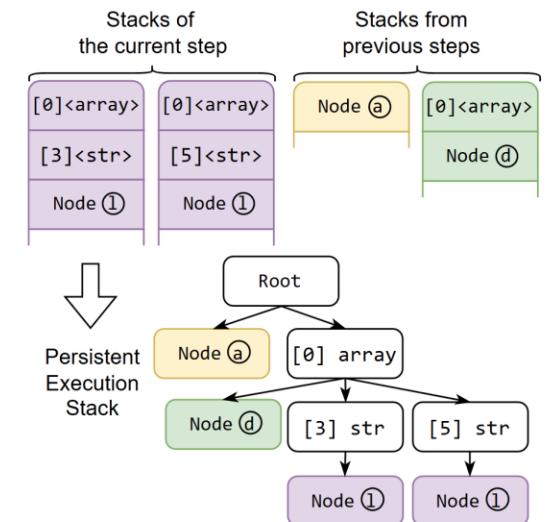


Token 'a' requires dynamic selection among states 5/6/7, increasing latency

Complex Runtime Stack Management

- Requires complex, tree-like stack tracking
- Large batches dramatically reduce decoding efficiency
- High resource demands for parallel stack management

Existing methods require complex tree-like stack management, increasing computational costs



- Pre³ focusing on LR(1) grammars enables transition disambiguation and unlocks a broader optimization surface.

LR(1) Grammar

- Powerful subset of context-free grammars (CFGs)
 - "L" stands for "Left-to-right scanning"
 - "R" stands for "Rightmost derivation in reverse," which is a fancy way of saying how it processes the rules.
 - "(1)" means it looks ahead only one token (like one word or one character) to figure out what to do next.
- LR(1) $\approx$ CFGs, but possess more elegant mathematical properties.
- An LR(1) grammar can be equivalently transformed into a DPDA.

PDAs and DPDAs

- Pushdown Automata(PDAs) is the powerful tools for processing grammars
 - States: Steps in rule execution (like recipe instructions)
 - Input: Generated text (processed token-by-token)
 - Stack Memory: A stack can be used to tracks nested rules
- Allow ambiguous transitions (multiple valid paths), which require
 - Backtracking on dead ends
 - Parallel path speculation
- Determined PDAs(DPDAs) are PDAs with always 1 unambiguous path and LR(1) compatible

(For more background in grammars, please refer to our paper)

Challenges and Contributions

Challenges

Leverage LR(1) information

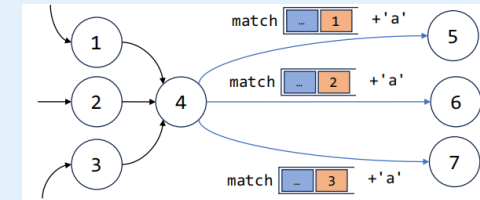
- Existing works overlook the inherent properties of LR(1) grammars
- LR(1) grammar can be transformed into a deterministic pushdown automaton

Inefficiency under large inference batches

- Baselines incurs up to 37.5% higher latency when processing batches of 512 requests

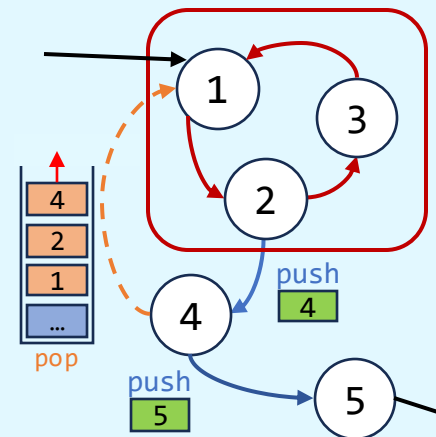
Our contributions

Prefix-conditioned Edges



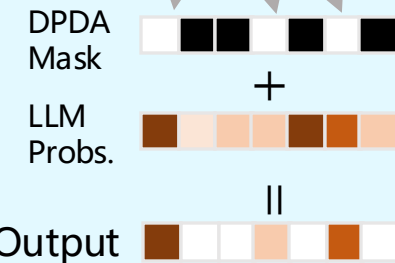
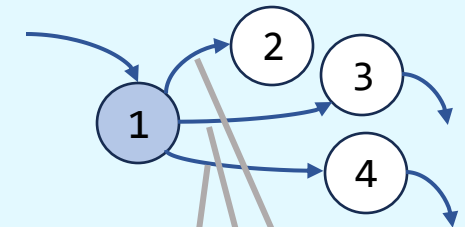
DPDA Construction

Applying prefix-conditioned edge



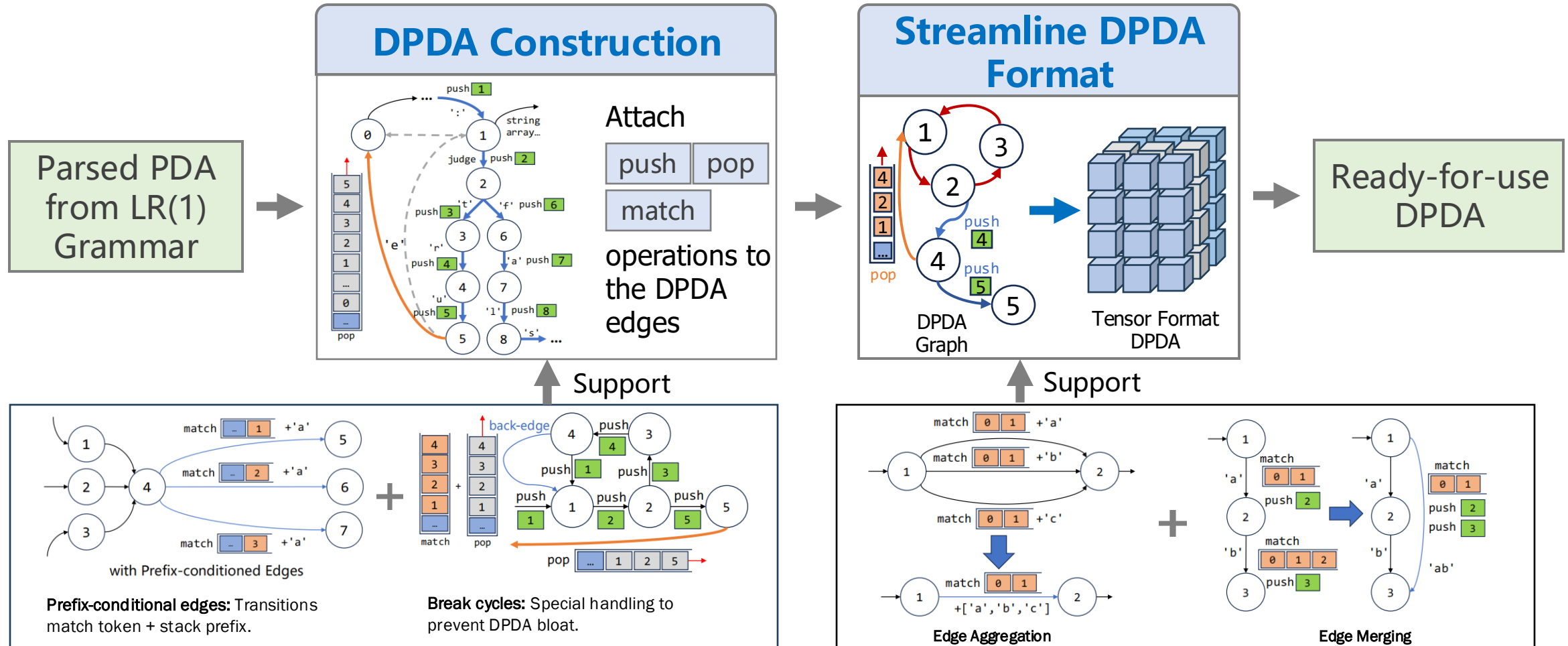
Break Cycles
Add Acceptance Edges
Add Reduction Edges

Efficient Generation



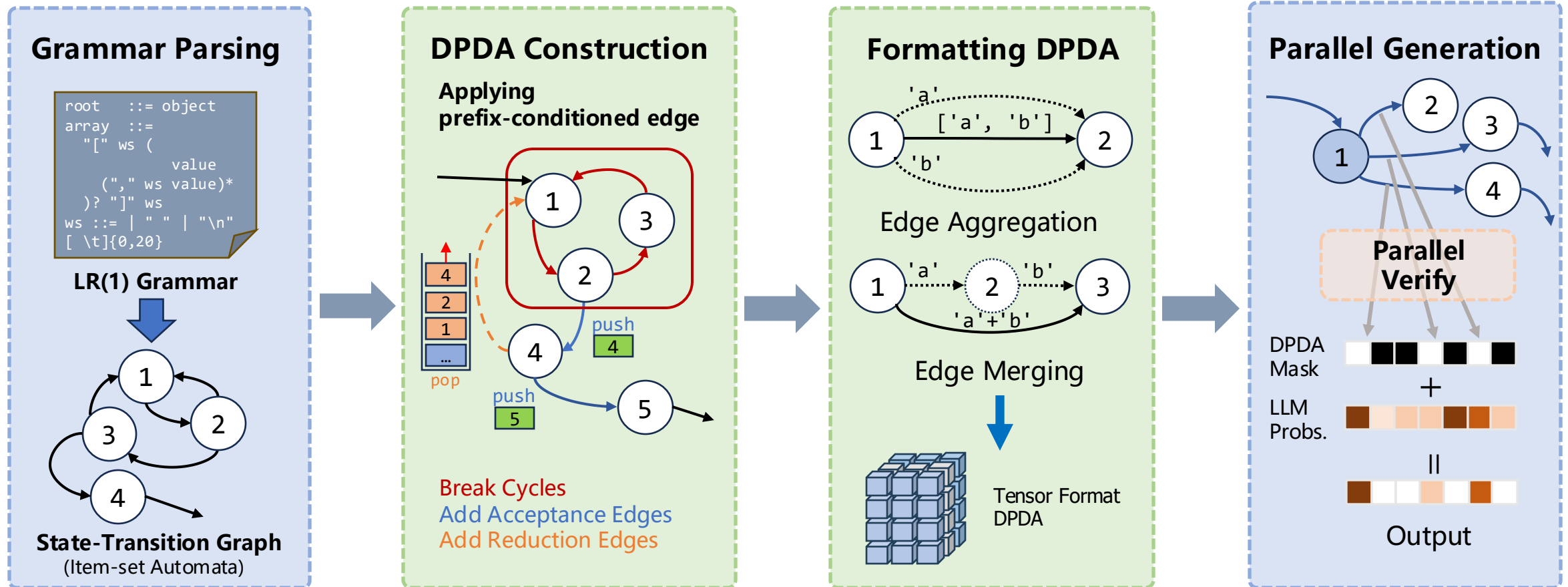
Key Idea

- Pre³ converts LR(1) grammar into a DPDA by precomputing transitions, replacing **runtime exploration** with efficient **table lookup**.



Pre³ Workflow

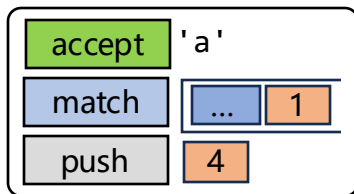
- Pre³ Workflow outlines our approach to convert LR(1) grammars into DPDAs by precomputing all transitions, enabling efficient table lookup over runtime exploration.



Prefix-conditioned Edges

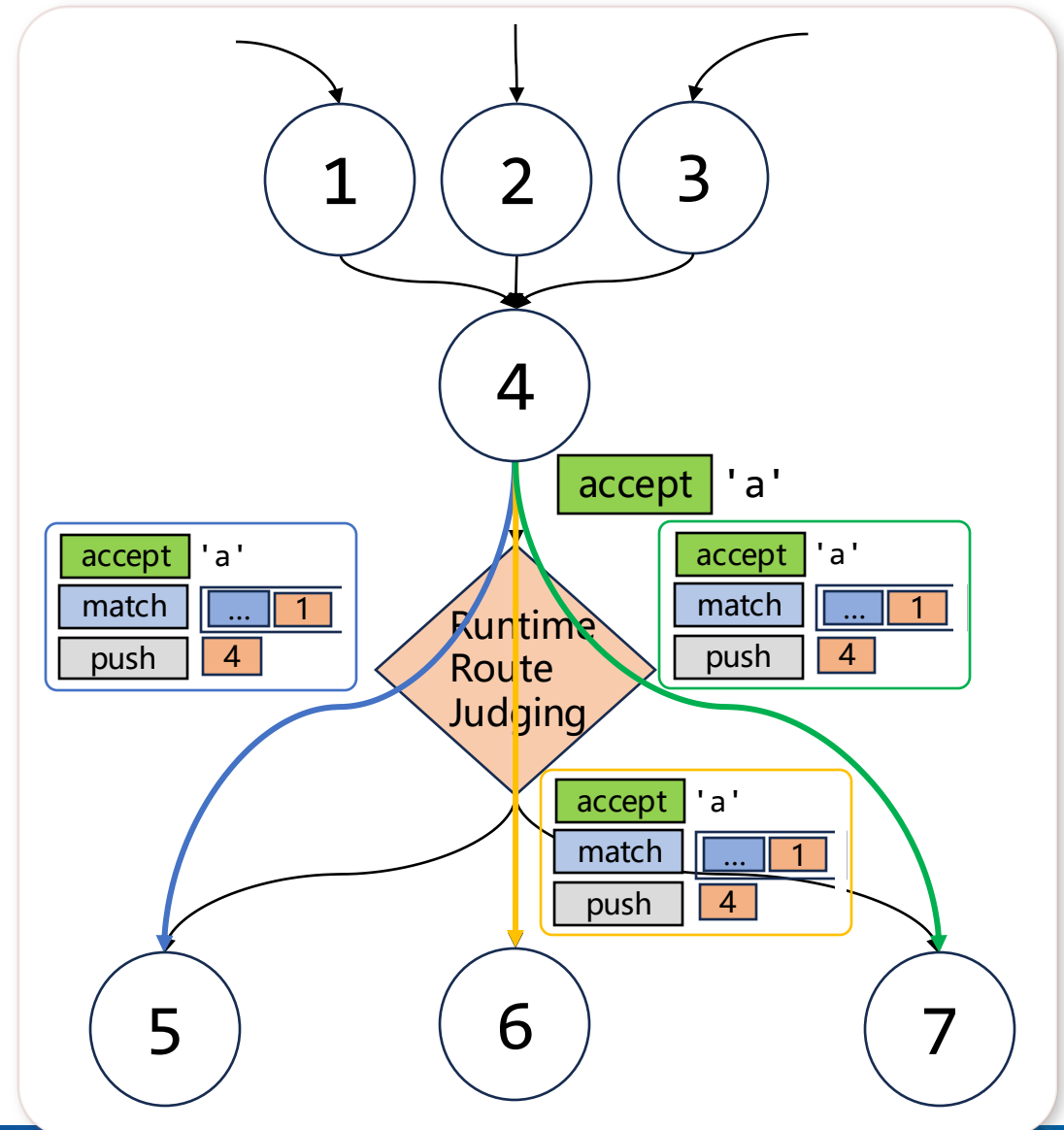
- **Problem:** Non-deterministic transitions require runtime stack/input checks → slow, complex
- **Solution:** Pre-computed transitions via stack prefix + input symbol → no runtime ambiguity

Prefix-conditioned Edges



- Accepted Symbol
- Stack Prefix Condition
- Stack Ops

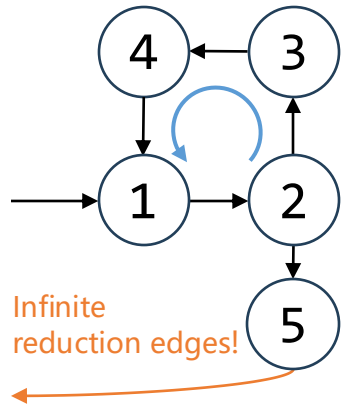
- ✓ **Faster:** Direct lookups
- ✓ **Simpler:** No dynamic resolution



Cycle-aware DPDA Construction

Break Cycles

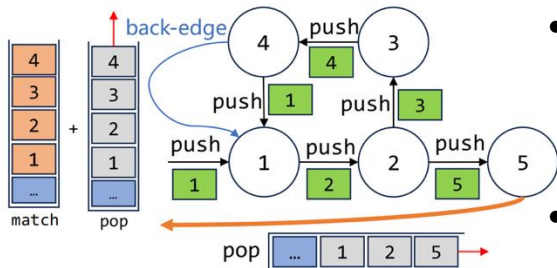
Cycles Cause Infinite Reduction



- Repeatedly reduce the same sequence of symbols (1-2-3-4-1)
- Creates an **infinite number of reduction edges**

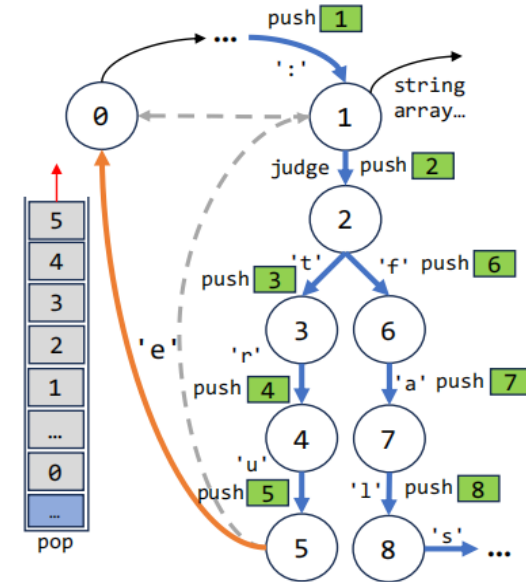


Add Back-edge to Resolve



- Add the **back-edge** ($4 \rightarrow 1$) using the match and pop operations
- Only **one reduction edge** is required.

Construct DPDA



Two types of DPDA Edges:

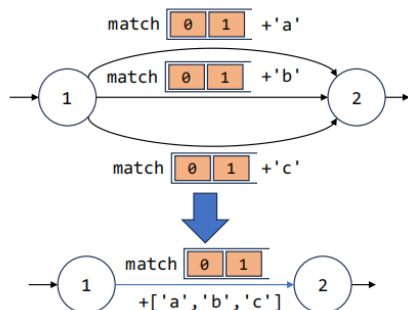
- **Acceptance Edges**
 - ← Represent shift operations
 - ← Push symbols to the stack
- **Reduction Edges**
 - ← Handle reductions
 - ← Pop symbols from the stack

Construction: Generated through graph traversal

Generation with DPDA

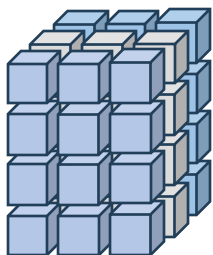
DPDA Formatting

Translate DPDA into Tensor

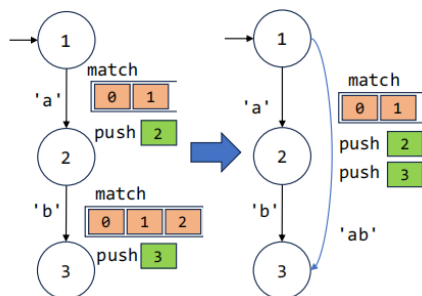


Edge Aggregation

Merge edges with identical stack conditions but different inputs (e.g., 'a'|'b'|'c' → [a-c]).



Tensor Format DPDA



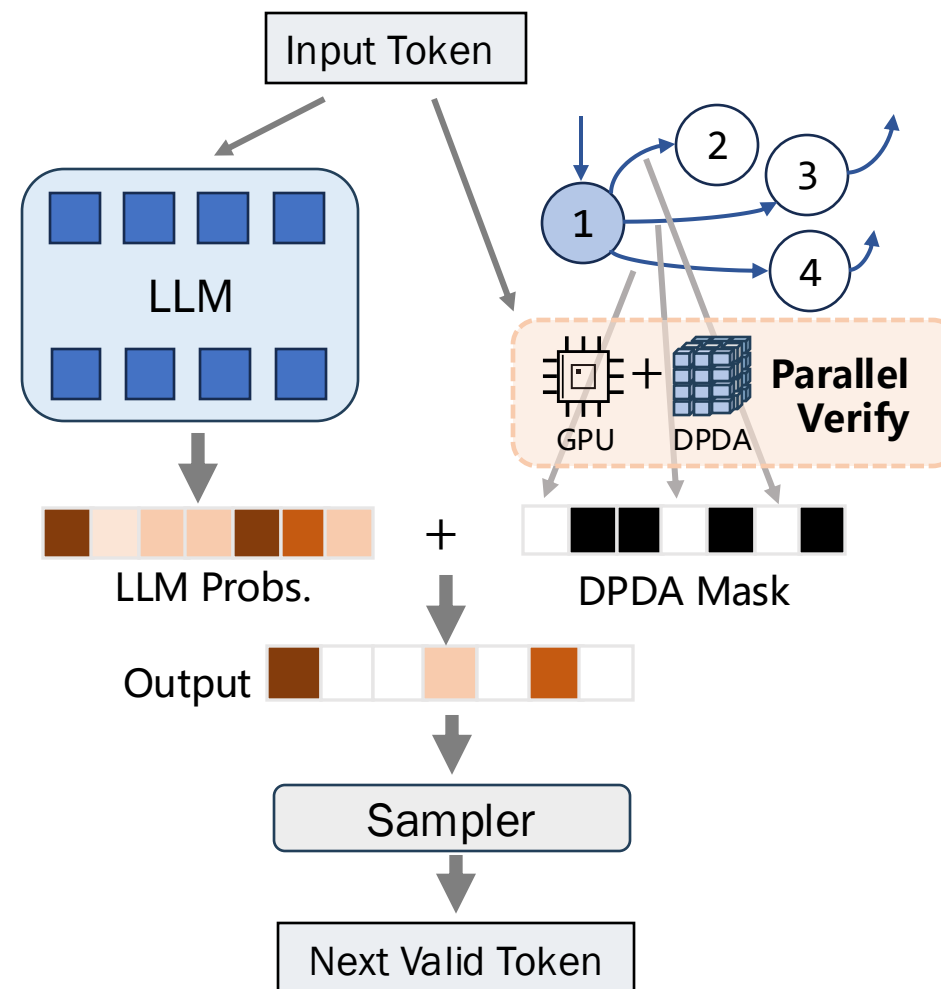
Edge Merging

Shortcut shared-condition edges to skip intermediate states.



Used For

Constrained Decoding



Experiments

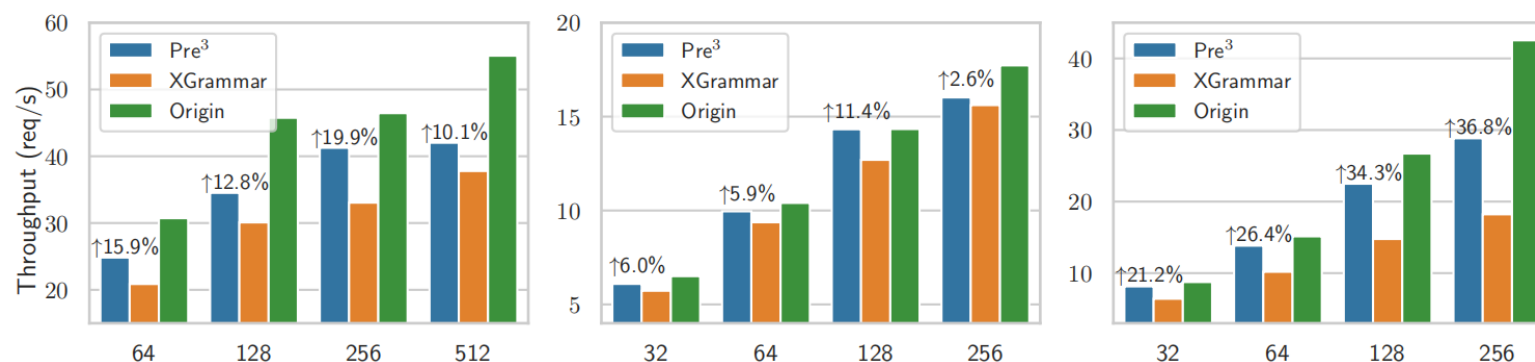
Evaluate batch decoding efficiency

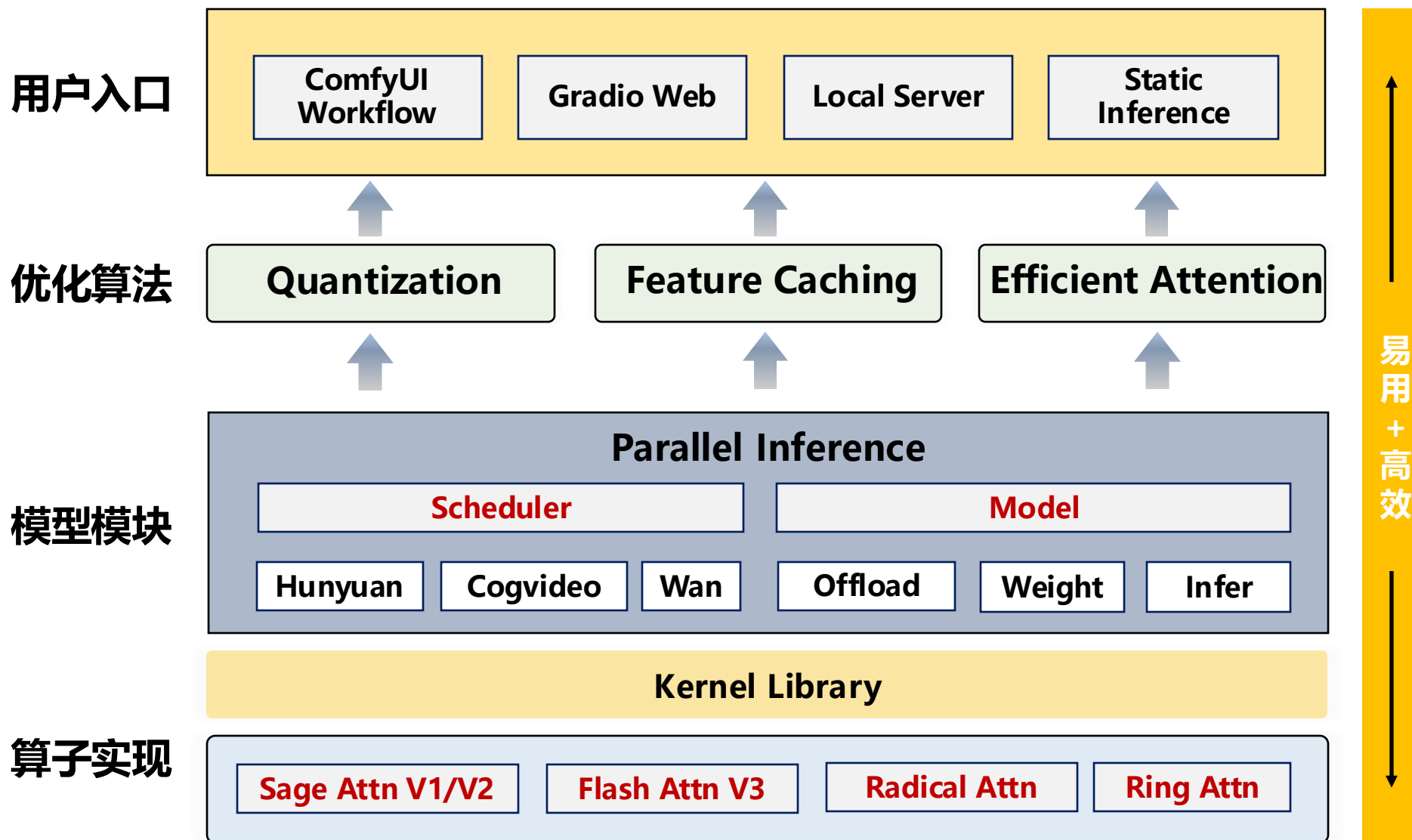
Max batch decode time comparison between Pre³ and baseline

Batch Size		16	32	64	128	256	512	1024
Llama-3-8B (Dubey et al., 2024) 2×H800	XGrammar (ms)	15.19	43.69	52.07	65.21	90.98	147.64	272.77
	Pre ³ (ms)	11.77	31.12	35.88	45.32	64.42	104.46	201.16
	Reduction	↓22.49%	↓28.78%	↓30.09%	↓30.50%	↓29.20%	↓29.24%	↓26.25%
DeepSeek-V2-Lite-Chat (Liu et al., 2024) 15.7B 2×H800	XGrammar (ms)	51.76	59.45	77.74	104.06	121.46	-	-
	Pre ³ (ms)	49.91	53.71	54.41	61.63	75.47	-	-
	Reduction	↓3.57%	↓9.65%	↓30.01%	↓40.78%	↓37.86%	-	-
Qwen2-14B (Yang et al., 2024a) INT8 2×H800	XGrammar (ms)	16.77	47.94	57.05	74.54	98.64	162.47	285.42
	Pre ³ (ms)	16.52	47.94	47.89	65.50	90.20	143.83	232.18
	Reduction	↓1.52%	↓0.12%	↓2.37%	↓12.14%	↓8.55%	↓11.47%	↓18.65%
Llama-2-70B (Touvron et al., 2023) 4×H800	XGrammar (ms)	28.75	55.12	56.94	68.79	85.92	-	-
	Pre ³ (ms)	27.20	54.24	54.18	62.27	75.72	-	-
	Reduction	↓5.39%	↓1.60%	↓4.85%	↓9.48%	↓11.87%	-	-

Benchmark under production-like serving conditions

System throughput based on different models and concurrency requests





• 全线INT8/FP8/FP4支持

Op	Dtype	Source
Linear	w8a8-int8	vLLM/SGLang
	w8a8-fp8	vLLM/SGLang/Q8-Kernel
	w8a8-fp8block	DeepGEMM
	w8a8-mx_fp8 w6a8-mx_fp6mx_fp8 w4a8-mx_fp4mx_fp8 w4a4-mx_fp4mx_fp4 w4a4-nvfp4	NV cutlass
Attention	int8 QK + int8 PV	Sage Attention
	int8/int4 QK + fp8 PV	Sage Attention2
	nvfp4 + nvfp4 PV	Sage Attention3
	fp8 QP + fp8 PV	Flash Attention3
	Sparse Token	Radial Attention, Sliding Tile Attention, Sparge Attention

- 在部分时间步跳过冗余计算，通过复用历史缓存结果提升推理效率

TeaCache

单卡H200推理耗时：58s



3.24x

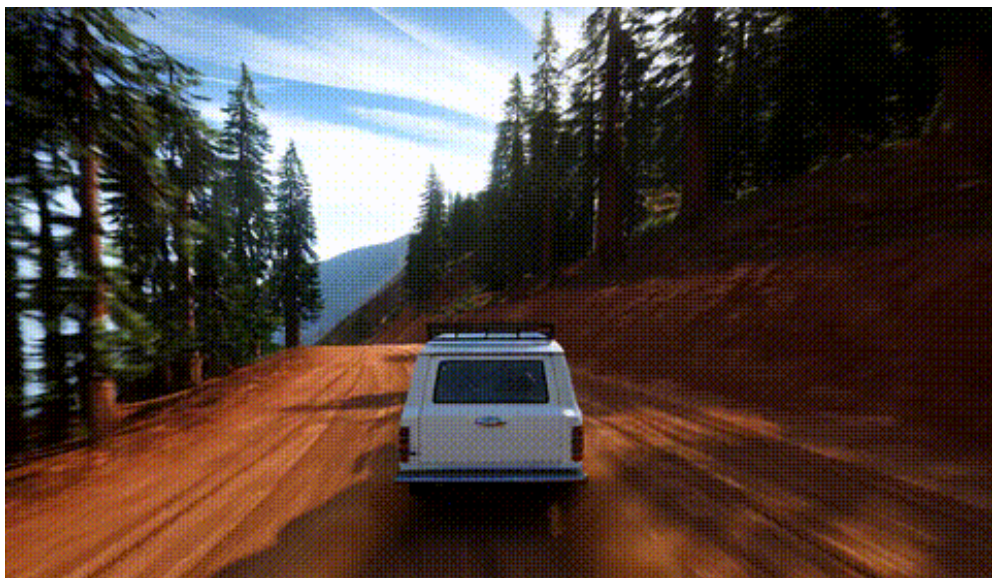
单卡H200推理耗时：17.9s



- 在部分时间步跳过冗余计算，通过复用历史缓存结果提升推理效率

TaylorSeer Cache

单卡H200推理耗时：57.7s



1.39x

单卡H200推理耗时：41.3s



- 在部分时间步跳过冗余计算，通过复用历史缓存结果提升推理效率

AdaCache

单卡H200推理耗时：227s



单卡H200推理耗时：83s

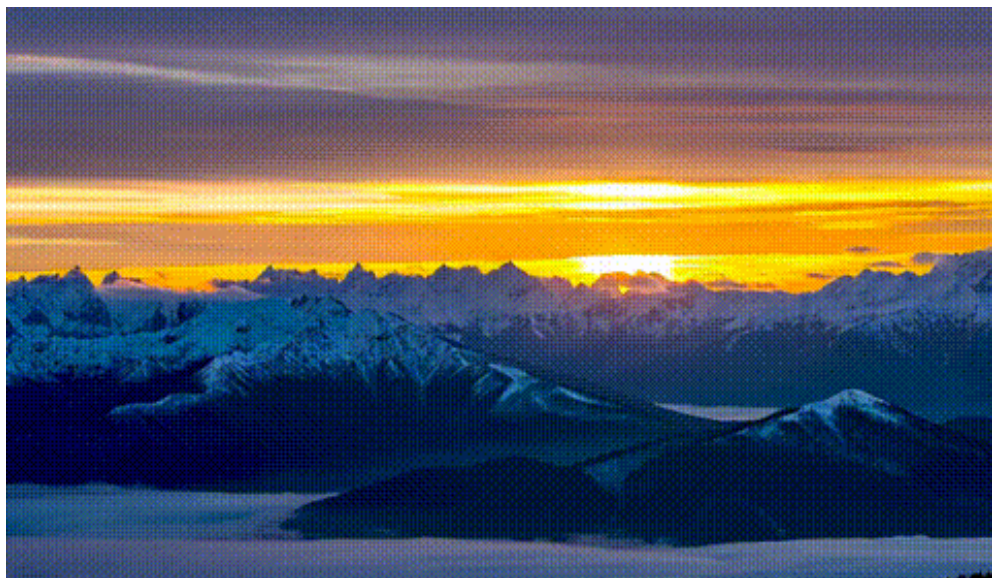


2.73x

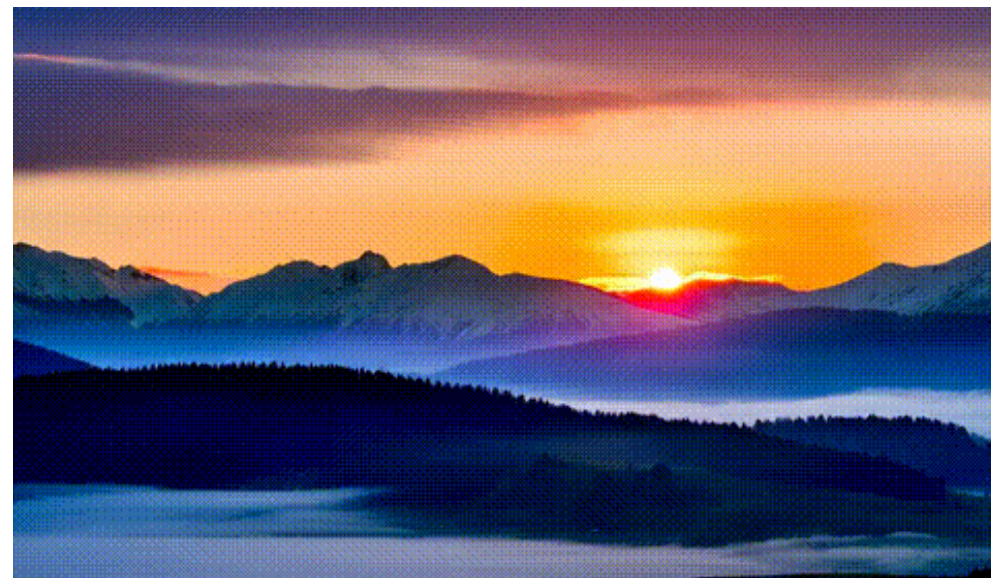
- 在部分时间步跳过冗余计算，通过复用历史缓存结果提升推理效率

CustomCache

单卡H200推理耗时：227s



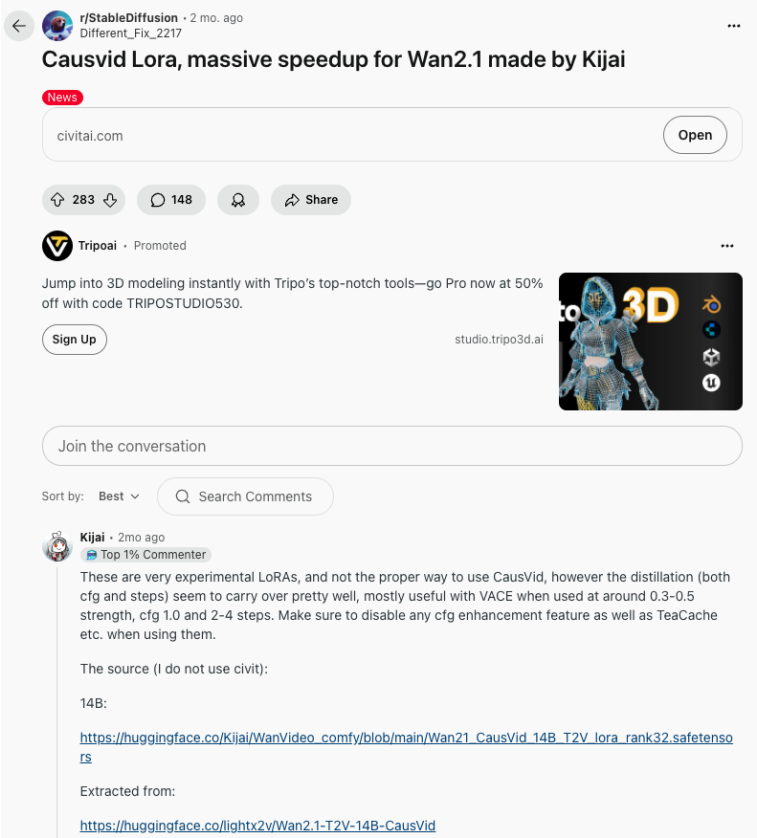
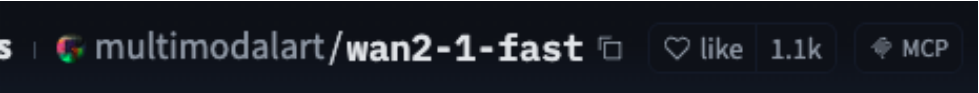
单卡H200推理耗时：83s



3.49x

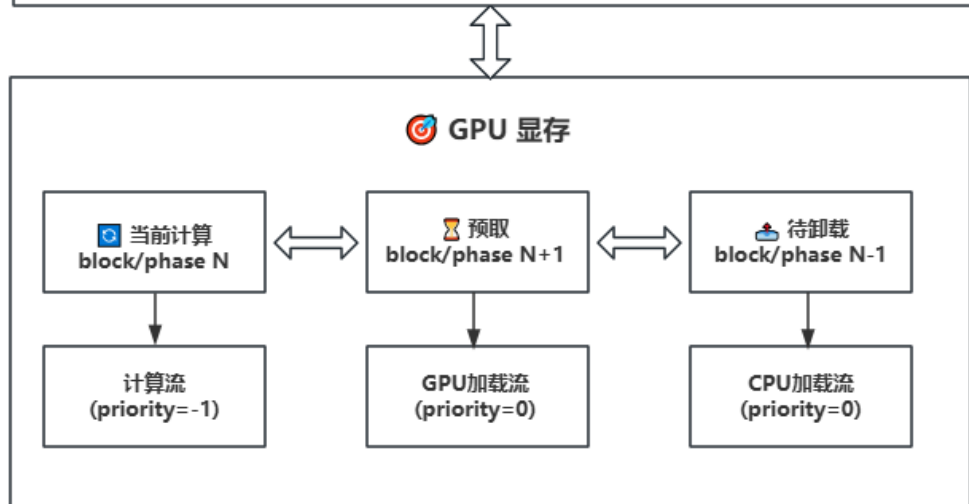
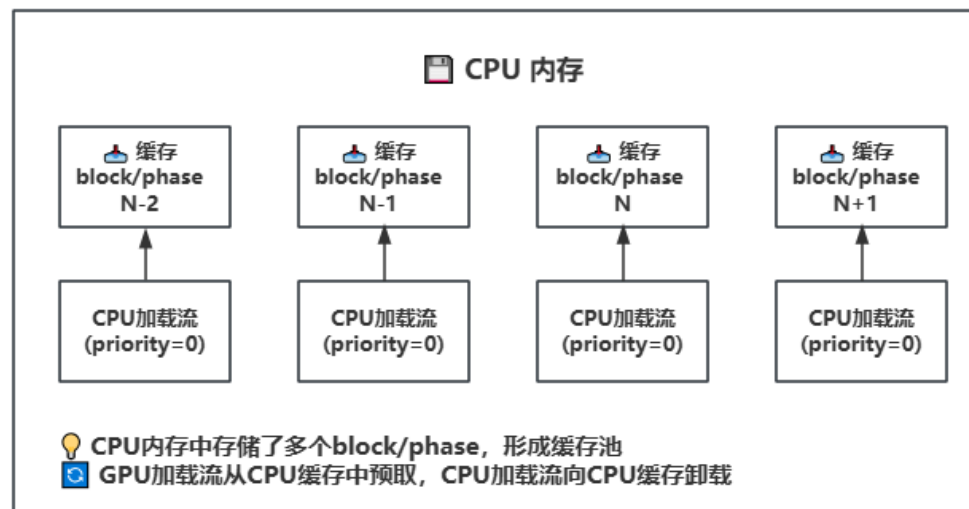
- 通过训练蒸馏模型将推理步数从原始的 40-50 步大幅减少到 4 步，在保持视频质量的同时显著提升推理速度，并进一步扩展CFG蒸馏

配置文件	用途
wan_t2v_distill_4step_cfg.json	加载 T2V 4步蒸馏完整模型
wan_i2v_distill_4step_cfg.json	加载 I2V 4步蒸馏完整模型
wan_t2v_distill_4step_cfg_lora.json	加载 Wan-T2V 模型和步数蒸馏 LoRA
wan_i2v_distill_4step_cfg_lora.json	加载 Wan-I2V 模型和步数蒸馏 LoRA



Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion. Arxiv 2025

• 显存不足内存充足：8G显存GPU可以跑起14B模型



💡 Swap 核心思想

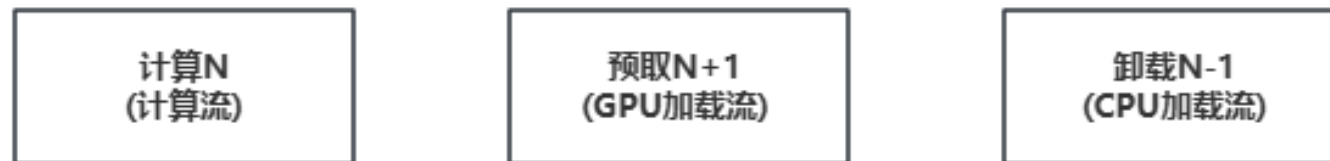
🔄 传统方式 vs Swap方式对比

传统方式



❌ 串行执行，存在等待时间，效率低

Swap方式

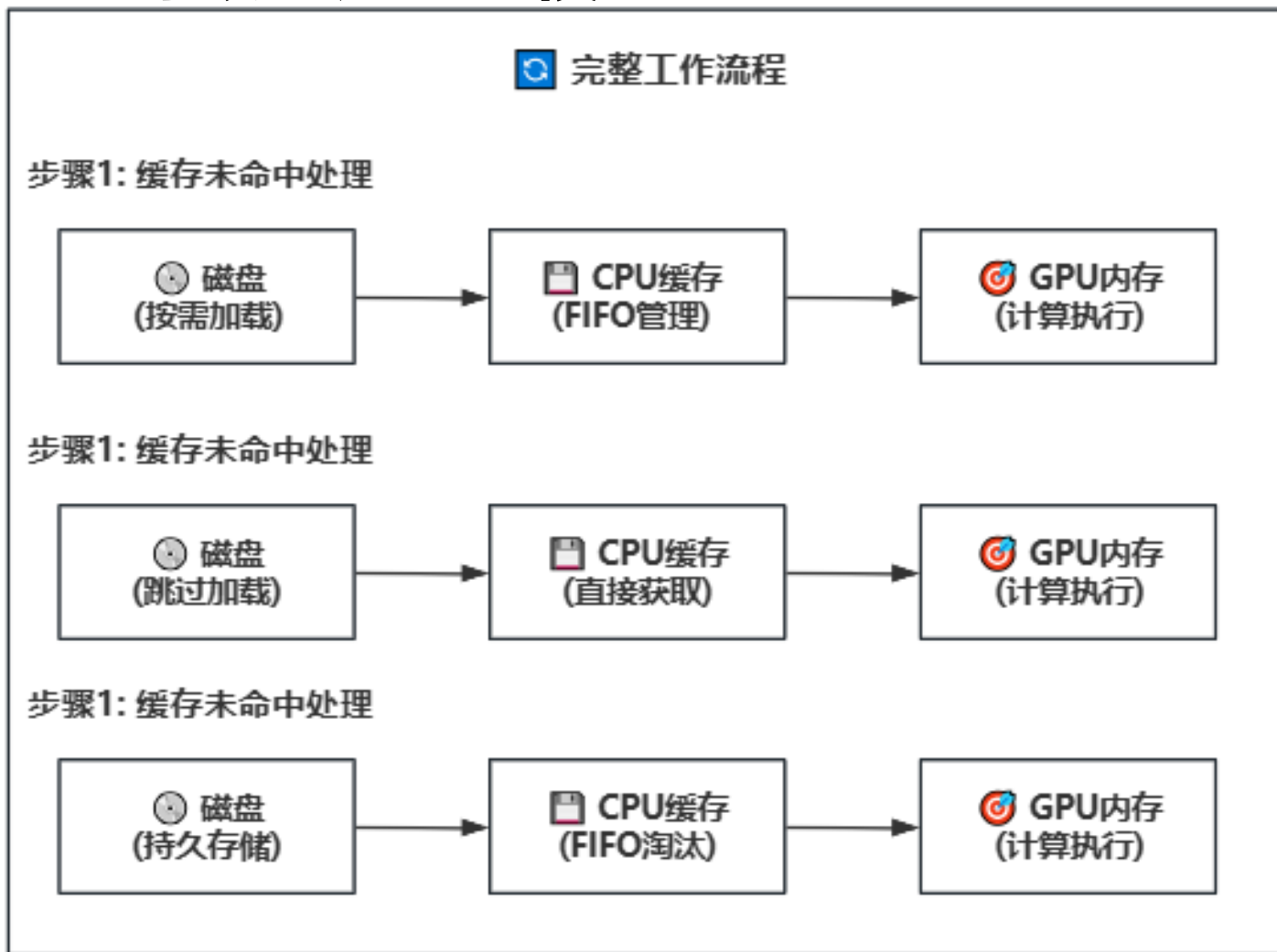
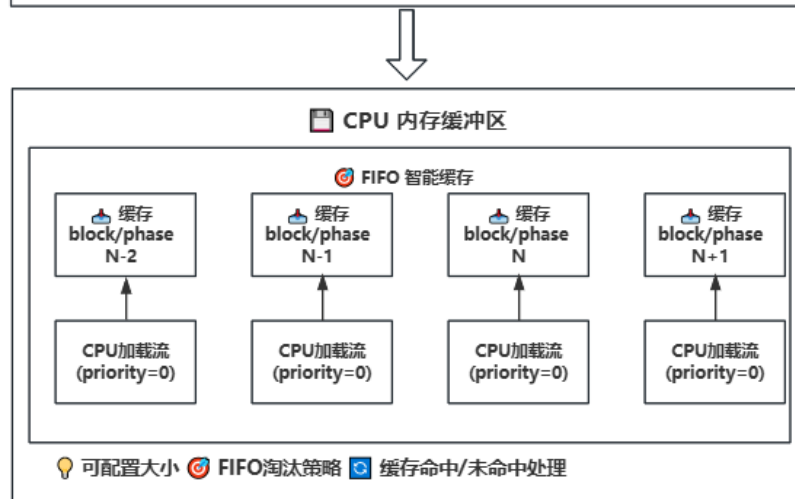
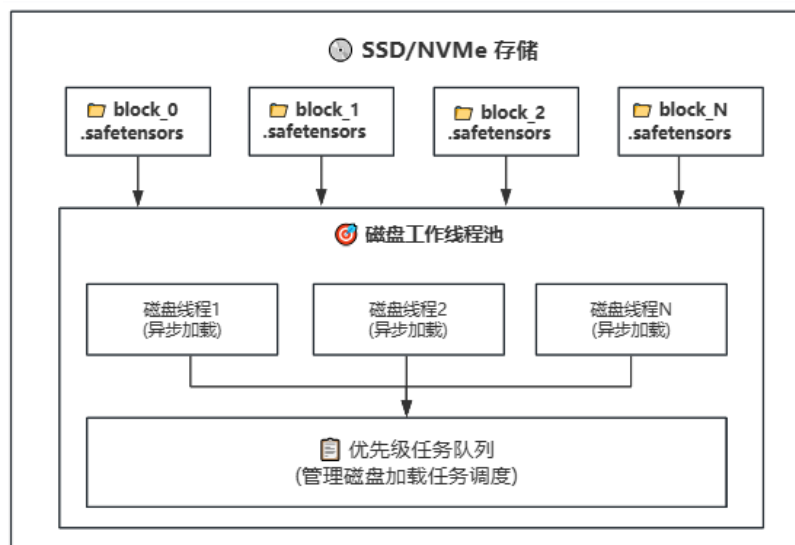


✅ 并行执行，无等待时间，效率高

🔄 Swap优势:

- 避免重复加载/卸载同一数据
- 通过位置轮换实现连续计算
- 最大化GPU利用率
- 减少内存碎片

• 显存不足内存不足：8G显存GPU可以跑起14B模型





推荐硬件规格

GPU要求:

- 显存: 8GB (RTX 3060/3070/4060/4060Ti 等)
- 架构: 支持CUDA的NVIDIA显卡

系统内存:

- 最低要求: 16GB DDR4
- 推荐配置: 32GB DDR4/DDR5
- 内存速度: 建议3200MHz及以上

存储要求:

- 类型: 强烈推荐NVMe SSD
- 容量: 至少50GB可用空间
- 速度: 读取速度建议3000MB/s以上

CPU要求:

- 核心数: 建议8核心及以上
- 频率: 建议3.0GHz及以上
- 架构: 支持AVX2指令集

量化方案对比

FP8量化 (推荐用于RTX 40系列):

```
# 适用于支持FP8的GPU, 提供更好的精度
dit_quant_scheme = "fp8"      # DIT模型量化
t5_quant_scheme = "fp8"      # T5文本编码器量化
clip_quant_scheme = "fp8"    # CLIP视觉编码器量化
```

INT8量化 (通用方案):

```
# 适用于所有GPU, 内存占用最小
dit_quant_scheme = "int8"     # 8位整数量化
t5_quant_scheme = "int8"     # 文本编码器量化
clip_quant_scheme = "int8"   # 视觉编码器量化
```

注意力算子选择

推荐优先级:

1. [Sage Attention](#) (最高优先级)
2. [Flash Attention](#) (通用方案)

矩阵乘算子选择

ADA架构显卡 (RTX 40系列):

推荐优先级:

1. [q8-kernel](#) (最高性能, 仅支持ADA架构)
2. [sglang-kernel](#) (平衡方案)
3. [vllm-kernel](#) (通用方案)

其他架构显卡:

1. [sglang-kernel](#) (推荐)
2. [vllm-kernel](#) (备选)

分块计算策略

旋转位置编码分块:

蒸馏模型 (强烈推荐):

- [Wan2.1-I2V-14B-480P-StepDistill-CfgDistill-Lightx2v](#)
- [Wan2.1-I2V-14B-720P-StepDistill-CfgDistill-Lightx2v](#)

VAE分块推理

- 标准VAE: 基准性能, 100%质量保持
- 标准VAE分块: 降低显存, 增加推理时间, 100%质量保持
- 轻量VAE: 极低显存, 视频质量有损

<https://github.com/ModelTC/LightLLM/>

<https://github.com/ModelTC/LightTTS/>

理解



LightLLM



文/图/音频->文

LightX2V



文/图/音频->视频

LightTTS



文本->音频

<https://github.com/ModelTC/LightX2V/>

LightKernel

<https://github.com/ModelTC/LightKernel/>

LightCompress

Quantization

Sparsification

Token Reduction

<https://github.com/ModelTC/LightCompress/>

生成



Thanks for Listening!